# ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

## EEM-272: Makine Öğrenmesine Giriş Raporu (Yazılı Doküman)

Dersin Öğretim Üyesi
Dr. Öğr. Üyesi Ali ŞENTÜRK

Raporu Yazan Öğrenci
*Enes AÇAR 2322705032*

**Giriş**

Makine öğrenimi 1959'ta bilgisayar oyunları ve yapay alanında öncü ve IBM çalışanı olan Amerikalı Arthur Samuel tarafından hayatımıza girmiştir [1]. Makine öğrenmesinde model eğitimi, bir algoritmanın girdi verileri ile hedef çıktılar arasındaki ilişkiyi temsil eden matematiksel parametreleri (weight ve bias) öğrenme sürecidir. Bu sürecin merkezinde yer alan optimizasyon, modelin tahminleri ile gerçek değerler arasındaki farkı ifade eden "maliyet fonksiyonunu" (cost function) minimize etme işlemidir. Teknik olarak optimizasyon; verinin modele girmeden önce ölçeklendirilmesi (scaling) , parametrelerin başlangıç değerlerinin atanması (initialization) , öğrenme hızı (learning rate) yönetimi ve gradyan inişi (gradient descent) temelli algoritmalar kullanılarak hatanın en düşük olduğu noktanın aranması süreçlerini kapsayan sistematik bir mühendislik yaklaşımıdır.

Bu konunun önem taşımasının temel nedenleri şunlardır:

1. Öğrenme Verimliliği ve Kararlılık: Girdi verilerinin normalize edilmesi, değişkenler arasındaki ölçüm farklılıklarını gidererek modelin büyüklük algısındaki yanılsamaları önler ve değişkenlere eşit şartlarda yaklaşılmasını sağlar.
2. Gradyan Problemlerinin Önlenmesi: Özellikle derin sinir ağlarında, türev büyüklüğünün hızla azalarak sıfıra yaklaşması (Vanishing Gradient) öğrenmeyi durdururken; hızla artarak sonsuza yaklaşması (Exploding Gradient) ağırlıkların kararsızlaşmasına neden olur. Bu da istenmeyen durumdur.
3. Ağırlık Başlatma Stratejileri: Ağırlıklara (w) rastgele veya 0 gibi yanlış değerler verilmesi, tüm nöronların aynı şeyi öğrenmesine veya gradyan inişi problemlerine yol açarak modelin kapasitesini sınırlar.
4. Hızlı ve Doğru Yakınsama: Adam ve RMSprop gibi gelişmiş optimizasyon algoritmaları, öğrenme oranını her parametre için otomatik ayarlayarak gürültüyü azaltır ve modelin minimum hata noktasına çok daha hızlı ve güvenilir bir şekilde ulaşmasını sağlar.

**Gradient Descent (Gradyan İnişi):** Makine öğrenmesi modellerinin "öğrenme" işlemini gerçekleştiren temel optimizasyon algoritmasıdır. Amacı, modelin yaptığı hataları temsil eden maliyet fonksiyonunu (cost function) en düşük seviyeye çekerek en doğru tahminleri yapacak parametreleri bulmaktır

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

w (ağırlık, weight) = Modelin veriyi ne kadar dikkate alacağını belirler.

b (bias) = Verinin modele daha iyi uyum sağlaması için kullanılır.

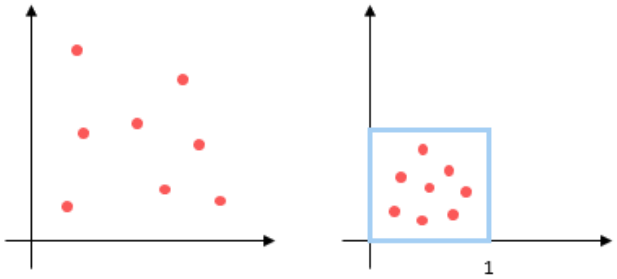
α (öğrenme oranı) = Adım büyüklüğünü belirleyen öğrenme oranıdır.

**Input Scaling (Girdi Ölçekleme):** Temel amacı veri setindeki farklı değişkenler arasındaki ölçüm farklılıklarını gidermektir böylece yanılsamaları önler.

**Normalization (Normalizasyon):** Veriyi 0 ile 1 arasına sıkıştırma işlemidir.

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Normalizasyon formülü yukarıdaki gibidir.




Burada normalizasyon uygulandığında verilerinin durumu gösterilmiştir. 0 ile 1 arasında sıkıştırılmıştır.

**Gradyan Problemleri**

Derin sinir ağlarında, eğitim süreci "Chain Rule" (Zincir Kuralı) prensibine dayanır; bu prensip gereği türevler son katmandan ilk katmana doğru çarpılarak ilerler. Bu süreçte iki temel problem ortaya çıkar:

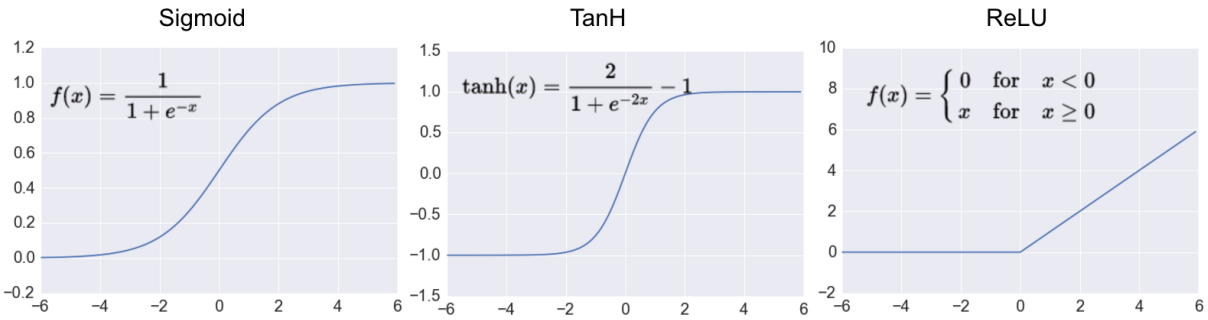
a. **Exploding Gradient (Patlayan Gradyan):** Katmanlar arasında geriye gidildikçe türev büyüklüğü hızla artar ve gradyan sonsuza yaklaşır. Bu durum, ağırlıkların (w) aşırı değişken hale gelmesine neden olarak verimli öğrenmeyi engeller.
b. **Vanishing Gradient (Kaybolan Gradyan):** Katmanlar arasında geriye gidildikçe türev büyüklüğü hızla azalır ve gradyan sıfıra yaklaşır. Gradyan çok küçüldüğü için ağırlıklar güncellenemez hale gelir ve bu da modelin öğrenme sürecini durdurur.

Şekilde gördüğümüz gibi exploding gradient'te sonsuza, vanishing gradient'te sıfıra gidiyor.

**Ağırlık Başlatma (Weight Initialization)**

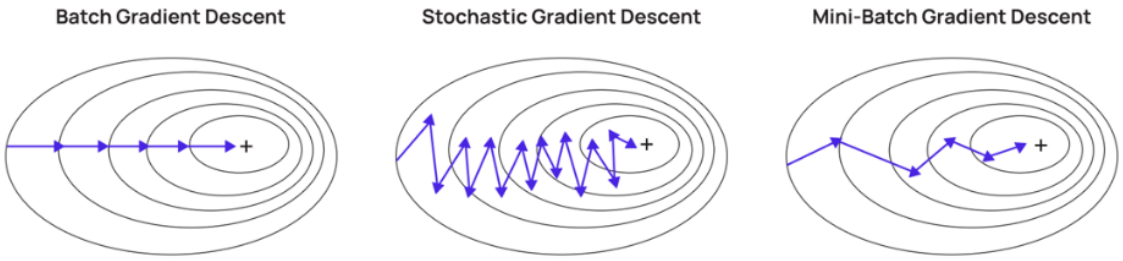
Ağırlık başlatma, model parametresi olan w değerine verilen ilk sayısal değerdir. Yanlış başlatma stratejileri modelin öğrenmesini imkansız kılar:

a. **Hatalı Başlatma:** Ağırlıklara 0 değeri verilirse tüm nöronlar aynı şeyi öğrenir; yanlış değer seçimi ise doğrudan gradyan problemlerine yol açar.
b. **Xavier (Glorot) Initialization:** Sigmoid ve Tanh aktivasyon fonksiyonları kullanıldığında tercih edilir. [2]
c. **He Initialization:** ReLU aktivasyon fonksiyonu kullanıldığında tercih edilir. [3]



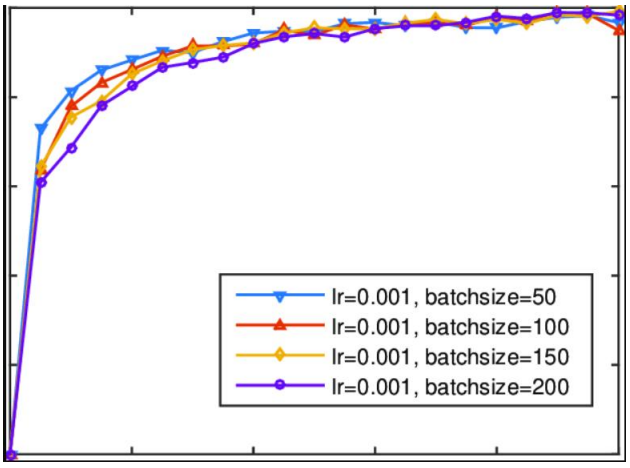

**Gradyan İnişi (Gradient Descent) Türleri:**

a. **Batch Gradient Descent:** Gradyan hesaplaması için tüm veri setini kullanır ve tek bir güncelleme yapar. Kararlı ve az gürültülüdür ancak büyük veri setlerinde hesaplama açısından çok maliyetli ve yavaştır.
b. **Mini-batch Gradient Descent:** Tüm veri setini küçük paketlere (batch) bölerek her paket sonunda güncelleme yapar. Batch yöntemine göre daha hızlıdır.
c. **Stochastic Gradient Descent (SGD):** Veri setinden rastgele seçilen tek bir örnek üzerinden güncelleme yapar. Çok hızlıdır ancak gürültülü güncellemeler yaparak zikzak çizerek ilerler.




**Learning Rate ve Batch Size**

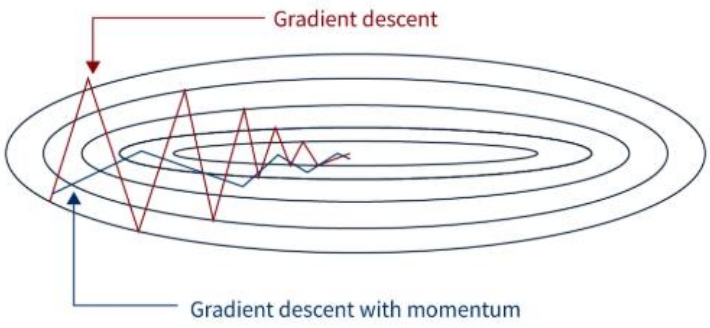
Modelin hedefe varış hızı ve kararlılığı bu iki parametreye bağlıdır: Batch size küçüldükçe ağırlık güncelleme sayısı artar. Küçük batch size'lar daha gürültülü olmasına rağmen, büyük batch size'lara kıyasla hedefe daha hızlı varabilir.



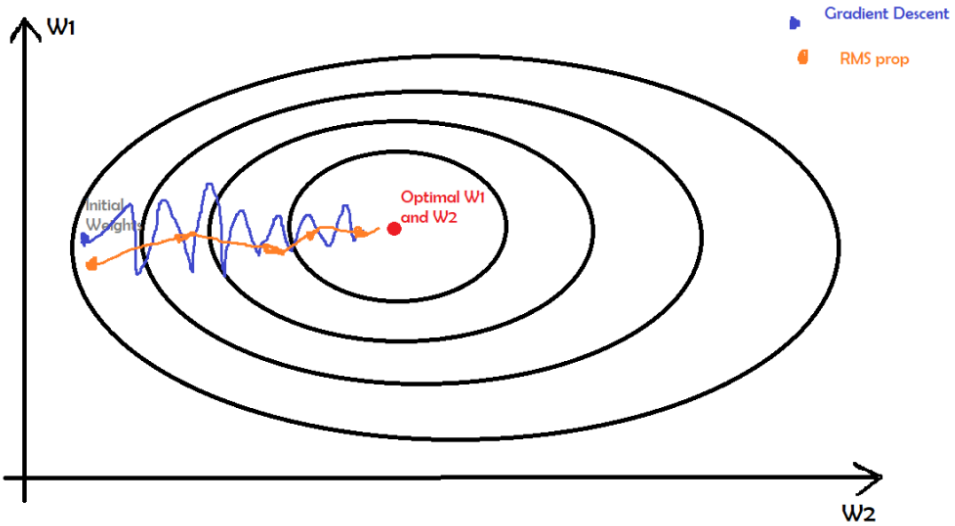
**5. Gelişmiş Optimizasyon Algoritmaları**

Standart gradyan inişinin eksikliklerini gidermek için geliştirilmiştir:

- **Momentum:** Geçmiş adımların hızını hatırlar. Doğru yönde ilerlerken adımları hızlandırır, yanlış yöndeki salınımları ve gürültüyü azaltır.




- **RMSprop:** Her bir parametre için öğrenme oranını otomatik ayarlar. Gradyan çok büyükse adımı küçültür, küçükse büyütür; böylece uygun bir hız yakalar.



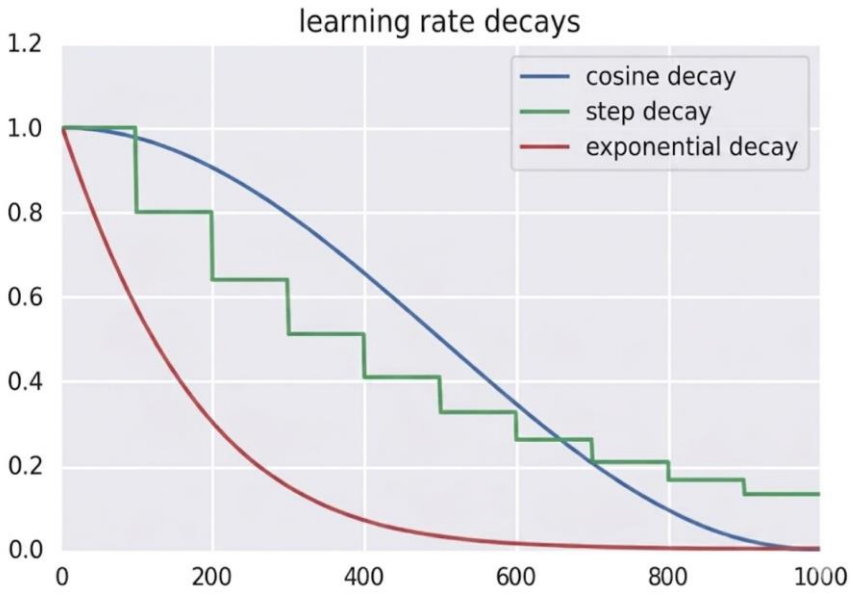

- **Adam (Adaptive Moment Estimation):** Momentum'un ivme kazandırma özelliği ile RMSprop'un otomatik hız ayarlama özelliğinin birleşimidir. [4]

**Learning Rate Scheduling (Öğrenme Oranı Programlama)**

Eğitim boyunca öğrenme oranını (α) sabit tutmak yerine belirli kurallarla azaltmak, modelin minimum noktasına daha hassas yaklaşmasını sağlar:

1. **Step Decay (Adımlı Azalma):** Öğrenme oranını belirli aralıklarla (epoch) sabit bir katsayıyla çarparak azaltır.
2. **Exponential Decay (Üstel Azalma):** Her adımda öğrenme oranını üstel olarak sürekli ve kademeli bir şekilde azaltır.

3. **Cosine Annealing (Kosinüs Azalması):** Öğrenme oranı, bir kosinüs dalgasının yarım periyodu şeklinde azalarak ilerler.




**Uygulama Alanları (Gerçek Dünyadan Örnekler)**

**Dijital Görüntü İşleme ve El Yazısı Tanıma:** Bu projenin de temelini oluşturan MNIST veri seti uygulaması, gerçek dünyada banka çeklerinin üzerindeki rakamların okunması, posta kodlarının otomatik olarak ayrıştırılması ve dijital formların sayısallaştırılması gibi alanlarda yaygın olarak kullanılır. Bu sistemlerde, piksellerin 0 ile 255 arasındaki ham değerlerinin normalize edilmesi, modelin farklı ışık koşullarında veya yazı tiplerinde daha tutarlı sonuçlar vermesini sağlar.

**Finansal Tahmin ve Ölçek Hassasiyeti Olan Veriler:** Input Scaling yönteminin temel amacı, değişkenler arasındaki ölçüm farklılıklarını gidererek modelin büyüklük yanılsamasına uğramasını engellemektir. Gerek dünya senaryolarında; bir ev fiyatı tahmini modelinde "oda sayısı" (küçük sayılar) ile "metrekare" veya "fiyat" (büyük sayılar) arasındaki uçurum, ölçeklendirme yapılmadığı takdirde modelin büyük sayılara gereksiz önem vermesine neden olur. Bu teknik, finansal piyasalardaki risk analizlerinde ve çok değişkenli sağlık verilerinin (nabız, yaş, kan değerleri) değerlendirilmesinde değişkenlere eşit şartlarda yaklaşılmasını sağlar.

**Büyük Veri ve Sosyal Medya Algoritmaları:** Milyonlarca kullanıcısı olan platformlarda tüm veri setini aynı anda işlemek (Batch Gradient Descent), hesaplama açısından aşırı maliyetli ve yavaştır.Mini-batch ve Stochastic Gradient Descent (SGD) algoritmaları, veriyi küçük paketler halinde işleyerek modellerin çok daha sık güncellenmesine olanak tanır. Bu durum, e-ticaret sitelerindeki öneri sistemlerinin veya sosyal medya akış algoritmalarının, kullanıcı davranışlarına anlık olarak uyum sağlamasına ve hızlı öğrenmesini sağlar.

**Otonom Sistemler ve İleri Seviye Optimizasyon:** Otonom araçlar veya gerçek zamanlı robotik sistemler gibi hataya yer olmayan alanlarda, modelin "minimum" hata noktasına en kararlı şekilde ulaşması hayati önem taşır. Adam gibi gelişmiş optimizasyon algoritmaları, gürültülü veriler içinde zikzaklar çizmek yerine geçmiş adımların hızını kullanarak doğru yöne odaklanır. Ayrıca, derin ağlarda karşılaşılan Vanishing Gradient problemi, "He" veya "Xavier" gibi doğru ağırlık başlatma yöntemleri ile aşılmaktadır. Bu sayede, çok katmanlı derin sinir ağları karmaşık nesne tespiti görevlerini başarıyla yerine getirebilmektedir. [6]

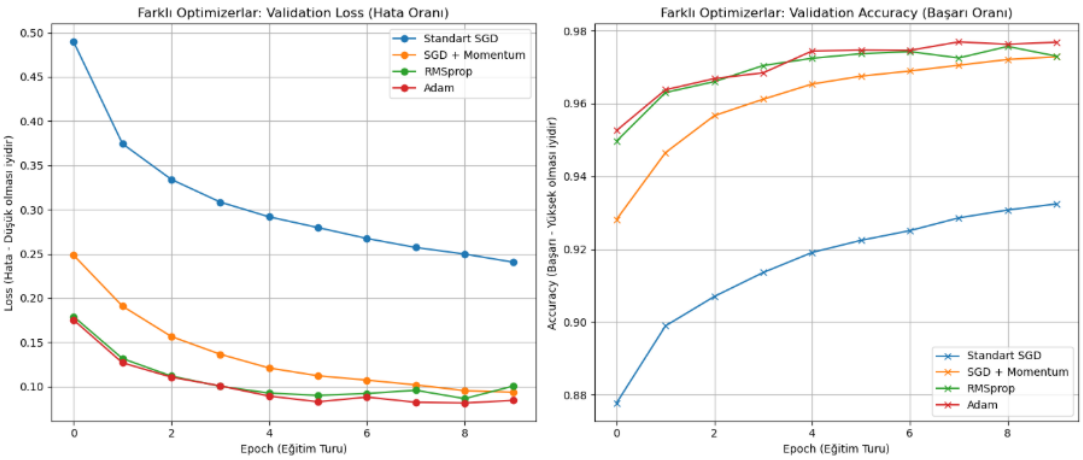
[2]:

```python
# %matplotlib inline: Jupyter Notebook'un grafikleri ayrı bir pencerede değil,
# doğrudan bu hücrenin altında çizmesini sağlayan komuttur.
#tensorflow: yz/derin öğrenme kütüphanesi
#matplotlib: grafik çizim kütüphanesi
%matplotlib inline

import tensorflow as tf
import matplotlib.pyplot as plt

# ====================================================================
# 1. VERİ YÜKLEME VE NORMALİZASYON (Sunum Madde 1: Input Scaling)
# ====================================================================
print("1. Veri seti yükleniyor ve normalizasyon yapılıyor...")

# MNIST veri setini indiriyoruz.
#train ile eğitiliyor, test ile test ediliyor
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Görüntü pikselleri normalde 0 ile 255 arasındadır.
# Modelin daha hızlı öğrenmesi ve "Gradient" problemlerini (exploding/vanishing gradyanlar)
# önlemek için tüm pikselleri 255'e bölerek 0 ile 1 arasına sıkıştırıyoruz (Scaling).
x_train, x_test = x_train / 255.0, x_test / 255.0


# ====================================================================
# 2. MODEL MİMARİSİ OLUŞTURMA (Sunum Madde 3: Weight Initialization)
# ====================================================================
def build_model():
    #sequential: Sıralı dizim
    model = tf.keras.models.Sequential([
        # 28x28 piksellik görüntüyü tek boyutlu (784 elemanlı) bir düz çizgiye çevirir.
        tf.keras.layers.Flatten(input_shape=(28, 28)),

        # 128 nöronlu gizli katman. 'relu' aktivasyon fonksiyonu kullanıyoruz.
        # Not: Keras varsayılan olarak "Glorot (Xavier) Uniform" Weight Initialization kullanır.
        #relu: negatif sayıları 0 yapan, pozitif sayılara dokunmayan aktivasyon fonk.
        tf.keras.layers.Dense(128, activation='relu'),

        # 10 farklı rakam (0-9) olduğu için çıkış katmanında 10 nöron var.
        #softmax fonksiyonu: çıkışları olasılık yüzdelerine (toplamı 1 olacak şekilde) çevirir.
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    return model
```

```python
# ====================================================================
# 3. OPTİMİZER (İYİLEŞTİRİCİ) LİSTESİ (Sunum Madde 6: Gelişmiş Algoritmalar)
# ====================================================================
optimizers = {
    # Klasik Yöntem: Sadece anlık eğime (gradyan) bakar. Yavaştır. Bu yüzden lr'i yüksek tuttuk.
    'Standart SGD': tf.keras.optimizers.SGD(learning_rate=0.01),

    # Momentum: SGD'ye fiziksel bir ivme katar. Önceki adımların hızını hatırlar,
    # böylece takılmadan daha hızlı ilerler.
    'SGD + Momentum': tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),

    # RMSprop: Her ağırlık için öğrenme oranını (learning rate) otomatik ayarlar.
    'RMSprop': tf.keras.optimizers.RMSprop(learning_rate=0.001),

    # Adam: Hem Momentum'un ivmesini hem de RMSprop'un otomatik hız ayarını birleştirir.
    'Adam': tf.keras.optimizers.Adam(learning_rate=0.001)
}


# ====================================================================
# 4. EĞİTİM (TRAINING) AŞAMASI (Sunum Madde 4 & 5: Mini-batch)
# ====================================================================
history_dict = {}
epochs = 10 # Model veriyi 10 kez baştan sona görecek.

# Sunum Madde 4-5: Tüm veriyi tek seferde vermek RAM'i şişirir ve verimsizdir.
# Bu yüzden veriyi 64'lük paketler (Mini-batch) halinde veriyoruz.
batch_size = 64

print("\n--- 2. Modellerin Eğitimi Başlıyor ---")

# Yukarıda tanımladığımız her bir optimizer için döngü başlatıyoruz.
for name, opt in optimizers.items():
    print(f">> {name} modeli eğitiliyor...")

    model = build_model() # Temiz, sıfır bir model çağırıyoruz.

    # Modeli derliyoruz: Loss fonksiyonu hataları hesaplar, Optimizer bu hataları düzeltir.
    model.compile(optimizer=opt,
                  #Loss, modelin ne kadar kötü old. hesaplayan cezalandırma sistemi
                  loss='sparse_categorical_crossentropy', # Çok sınıflı sınıflandırma için standart hata ölçer
                  metrics=['accuracy']) # Bizim anlayacağımız %'lik başarı oranı

    # Model eğitimi başlıyor (fit)
    history = model.fit(x_train, y_train,
                        epochs=epochs,
                        batch_size=batch_size,
                        validation_split=0.2, # Verinin %20'sini test (doğrulama) için ayır.
                        verbose=0) #Ekrana binlerce satır log basmasını engelliyoruz. Yani sessizce çalışıyor.

    # Çizim yapabilmek için eğitim sonuçlarını (hata ve başarı oranlarını) kaydediyoruz.
    history_dict[name] = history.history
```

```python
# ======================================================================
# 5. SONUÇLARI GÖRSELLEŞTİRME (Proje Çıktısı)
# ======================================================================
print("\n--- 3. Eğitim Bitti! Grafikler Çiziliyor ---")

#14'e
plt.figure(figsize=(14, 6))

# Sol Grafik: Validation Loss (Doğrulama Hata Oranı)
plt.subplot(1, 2, 1)
for name, history in history_dict.items():
    plt.plot(history['val_loss'], label=name, marker='o') # Hata verilerini çizdir
plt.title('Farklı Optimizerlar: Validation Loss (Hata Oranı)')
plt.xlabel('Epoch (Eğitim Turu)')
plt.ylabel('Loss (Hata - Düşük olması iyidir)')
plt.legend()
plt.grid(True)

# Sağ Grafik: Validation Accuracy (Doğrulama Başarı Oranı)
#subplot(satır, sutun, sıra)
plt.subplot(1, 2, 2)
for name, history in history_dict.items():
    plt.plot(history['val_accuracy'], label=name, marker='x') # Başarı verilerini çizdir
plt.title('Farklı Optimizerlar: Validation Accuracy (Başarı Oranı)')
plt.xlabel('Epoch (Eğitim Turu)')
plt.ylabel('Accuracy (Başarı - Yüksek olması iyidir)')
plt.legend()
plt.grid(True)

plt.tight_layout() # Grafikler birbirine girmesin diye düzenler
plt.show() # Grafiği ekrana bas
```

```
--- 2. Modellerin Eğitimi Başlıyor ---
>> Standart SGD modeli eğitiliyor...
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be
used. Please use WSL2 or the TensorFlow-DirectML plugin.
C:\Users\DELL\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input_shape`/`inpu
t_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
>> SGD + Momentum modeli eğitiliyor...
>> RMSprop modeli eğitiliyor...
>> Adam modeli eğitiliyor...

--- 3. Eğitim Bitti! Grafikler Çiziliyor ---
```




[ ]:

**Yapılan Mini Projenin Amacı:** Farklı optimizasyon algoritmalarının (SGD, Momentum, RMSprop, Adam) modelin öğrenme hızı ve başarısı üzerindeki etkilerini karşılaştırmaktır.

**Yapılan Çalışmanın Açıklaması**: Veriler, donanımı (RAM) yormamak ve modeli daha verimli eğitmek adına 64'lük mini-batch kullanılmıştır. Kurulan bu mimari, dört farklı optimizasyon algoritması kullanılarak sıfırdan 10'ar tur (epoch) eğitilmiştir. Her bir algoritmanın eğitim sırasındaki başarım ve hata oranları kaydedilmiş, son aşamada ise bu sonuçlar birbiriyle kıyaslanabilmesi için grafiklere dökülmüştür.

**Kullanılan Veri Seti:** Projede MNIST(el yazısı rakamları (0-9) tanımayı öğretmek için kullanılan devasa bir resim arşivi) veri seti kullanılmıştır. [5]

**Kullanılan Yöntem:** Normalizasyon kullanılarak görüntü pixelleri 0-255 aralığından 0-1 aralığına sıkıştırılarak gradyan problemlerinin önüne geçilmiştir. Aktivasyon fonksiyonları olarak ara katmanda negatif değerleri sıfırlayan ReLU, çıkış katmanında ise sonuçları toplamı 1 olacak şekilde yüzdelik olasılıklara dönüştüren Softmax kullanılmıştır. Son olarak model ezberlemesin (overfitting) diye, eğitim verisinin %20'si doğrulama (validation_split) ile önüne geçilmiştir.

**Kullanılan Kodlamalar**

Python Kütüphaneleri: * tensorflow.keras: Sinir ağı mimarisini kurmak, katmanları oluşturmak ve eğitimi gerçekleştirmek için kullanılmıştır.

matplotlib.pyplot: Eğitim bittikten sonra sonuçların görselleştirilmesi (grafik çizimi) için kullanılmıştır.

Kayıp Fonksiyonu (Loss Function): Çok sınıflı sınıflandırma (0-9 arası 10 sınıf) yapıldığı için sparse_categorical_crossentropy tercih edilmiştir.

**Elde Edilen Sonuçlar ve Program Çıktıları**

Programın nihai çıktısı, ekranın altında yan yana açılan iki adet grafiktir. Bu grafikler algoritmaların performansını gösterir:

Sol grafikte (validation loss - hata oranı): Standart SGD algoritmasının hata oranını yavaş yavaş düşürdüğü görülürken; Momentum, RMSprop ve özellikle Adam algoritmalarının hatayı çok daha hızlı bir şekilde sıfıra yaklaştırdığı gözlemlenir.

Sağ grafikte (validation accuracy -başarı oranı): Modelin rakamları doğru bilme yüzdesini gösterir. SGD en düşük başarıda kaldığı ve yavaş yükseldiği görülürken; diğer gelişmiş algoritmaların (RMSprop, Adam) yüksek bir tahmin başarısına ulaştığı grafikte gözükmektedir.

**KAYNAKÇA:**


[1] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210-229, 1959. [Çevrimiçi]. Erişim: https://ieeexplore.ieee.org/document/5392560

[2] X. Glorot ve Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* içinde, 2010, pp. 249-256. [Çevrimiçi]. Erişim: https://proceedings.mlr.press/v9/glorot10a.html

[3] K. He, X. Zhang, S. Ren ve J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* içinde, 2015, pp. 1026-1034. [Çevrimiçi]. Erişim: https://arxiv.org/abs/1502.01852

[4] D. P. Kingma ve J. Ba, "Adam: A method for stochastic optimization," *International Conference on Learning Representations (ICLR)* içinde, 2015. [Çevrimiçi]. Erişim: https://arxiv.org/abs/1412.6980

[5] Y. LeCun, C. Cortes ve C. J. Burges. "The MNIST database of handwritten digits." (1998). [Çevrimiçi]. Erişim: https://cir.nii.ac.jp/crid/1571417126193283840

[6] Y. LeCun, Y. Bengio ve G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436-444, Mayıs 2015. [Çevrimiçi]. Erişim: https://www.nature.com/articles/nature14539